

SIMATS ENGINEERING

ASSESSMENT TOOL 1

COURSE NAME: INTERNET PROGRAMMING
COURSE CODE: CSA4301

COURSE OUTCOME COVERED

CO 3: Implement dynamic web applications by utilizing DOM-based client-side scripting and employing Java Servlets for server-side request processing and response handling. (L3)

Assessment Tool : Scenario-Based Assessment

Weightage: 40%

QUESTIONS:

Total Marks: 30

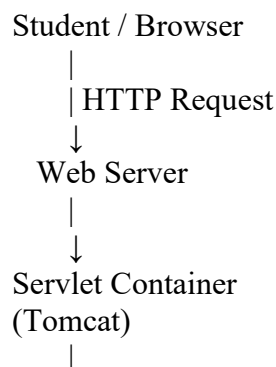
Question 1

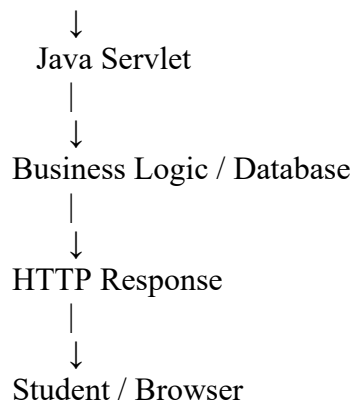
Suppose that a college wants to develop an online student registration system where students submit their details through a web form, and the data is processed using a Java Servlet.

Explain the architecture of a servlet-based application. Design and implement a servlet to handle form data using both GET and POST methods. Describe the servlet life cycle (init, service, destroy) and justify which method is more appropriate for handling sensitive data.

1. Servlet-Based Application Architecture

A **Servlet** is a Java program that runs on a web server and handles requests from clients, processes data, and generates a response.





Working

1. The student opens the registration webpage.
2. The student enters details such as name, register number, email, and course.
3. The browser sends an HTTP request to the servlet.
4. The servlet container, such as **Apache Tomcat**, receives the request.
5. The container identifies the appropriate servlet.
6. The servlet processes the submitted information.
7. If required, the servlet communicates with a database or business-logic layer.
8. The servlet generates an HTTP response.
9. The response is returned to the student's browser.

HTML:

```
<form action="StudentServlet" method="post">
  Name: <input type="text" name="name"><br>
  Register No: <input type="text" name="regno"><br>
  Course: <input type="text" name="course"><br>
  <input type="submit" value="Register">
</form>
```

SERVELT USING GET AND POST:

```
import java.io.*;
import javax.servlet.*;
import javax.servlet.http.*;

public class StudentServlet extends HttpServlet {

    protected void doGet(HttpServletRequest req,
                          HttpServletResponse res)
        throws IOException {

        String name = req.getParameter("name");

        res.setContentType("text/html");
        PrintWriter out = res.getWriter();

        out.println("Student Name: " + name);
```

```

    }

    protected void doPost(HttpServletRequest req,
                           HttpServletResponse res)
                           throws IOException {

        String name = req.getParameter("name");
        String regno = req.getParameter("regno");
        String course = req.getParameter("course");

        res.setContentType("text/html");
        PrintWriter out = res.getWriter();

        out.println("Registration Successful<br>");
        out.println("Name: " + name + "<br>");
        out.println("Register No: " + regno + "<br>");
        out.println("Course: " + course);
    }
}

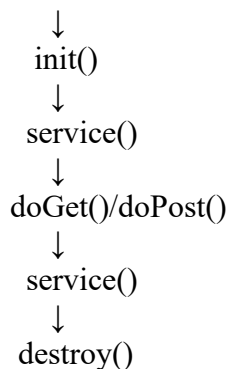
```

Servlet Life Cycle

There are three main methods:

1. `init()`
Called only once when the servlet is created. It is used for initialization.
2. `service()`
Called for every request. It decides whether to call `doGet()` or `doPost()`.
3. `destroy()`
Called once before the servlet is removed from the server. It is used to release resources.

Create Servlet



GET vs POST

GET	POST
Data appears in URL	Data is sent in request body
Less secure for sensitive data	Better for sensitive form submission

Used mainly for retrieving data

Used mainly for submitting data

Question 2

Suppose that a website requires users to log in and maintain their session across multiple pages after authentication.

Design a session management system using HttpSession and Cookies in Servlets. Explain how session tracking works in web applications and compare session-based tracking with cookies.

Introduction

HTTP is a **stateless protocol**, meaning the server does not automatically remember a user between different requests.

Session tracking is used to remember users after login.

HttpSession

HttpSession stores user information on the server.

Login Example

```
import javax.servlet.*;
import javax.servlet.http.*;
import java.io.*;
```

```
public class LoginServlet extends HttpServlet {

    protected void doPost(HttpServletRequest req,
                          HttpServletResponse res)
        throws IOException {

        String username = req.getParameter("username");
        String password = req.getParameter("password");

        if(username.equals("student") &&
           password.equals("1234")) {

            HttpSession session = req.getSession();

            session.setAttribute("username", username);

            res.sendRedirect("home");
        }
        else {
            res.getWriter().println("Invalid Login");
        }
    }
}
```

Accessing Session

```
HttpSession session = request.getSession(false);
```

```
String name =
    (String) session.getAttribute("username");
```

```
response.getWriter().println("Welcome " + name);
```

Logout

```
session.invalidate();
```

This removes the session.

Cookies

A cookie is a small piece of information stored in the user's browser.

Creating a Cookie

```
Cookie c = new Cookie("username", "student");
```

```
response.addCookie(c);
```

Reading a Cookie

```
Cookie[] cookies = request.getCookies();
```

```
for(Cookie c : cookies) {  
  
    if(c.getName().equals("username")) {  
  
        String name = c.getValue();  
  
        response.getWriter().println(name);  
    }  
}
```

Session Tracking

User Login



Server creates Session



Session ID



Browser stores Session ID



Browser sends Session ID



Server identifies the user

HttpSession vs Cookies

HttpSession	Cookies
Data mainly stored on server	Data stored in browser
More suitable for login information	Suitable for small information
Session ID identifies the user	Cookie stores name/value
Can expire after session timeout	Can have an expiry time

.

Question 3

Suppose that an e-commerce platform needs to store and retrieve product details from a database using a Java-based web application.

Design a JDBC-based application to connect to a database, insert product data, and retrieve it using SQL queries. Explain the JDBC architecture and describe the steps involved in database connectivity along with proper exception handling.

JDBC (Java Database Connectivity) is used to connect a Java application with a database.

JDBC Architecture

Java Application



JDBC API



JDBC Driver



Database

Steps in JDBC

1. Load the JDBC driver.
2. Establish database connection.
3. Create SQL statement.
4. Execute SQL query.
5. Process the result.
6. Close the connection.

Product Table

```
CREATE TABLE product (  
    id INT,  
    name VARCHAR(50),  
    price DOUBLE  
);
```

Insert Product

```
import java.sql.*;
```

```
public class Product {  
  
    public static void main(String[] args) {  
  
        try {  
  
            Class.forName("com.mysql.cj.jdbc.Driver");  
  
            Connection con =  
                DriverManager.getConnection(  
                    "jdbc:mysql://localhost:3306/shop",  
                    "root",  
                    "root");  
  
            PreparedStatement ps =  
                con.prepareStatement(  

```

```

        "INSERT INTO product VALUES (?, ?, ?)");

ps.setInt(1, 101);
ps.setString(2, "Keyboard");
ps.setDouble(3, 800);

ps.executeUpdate();

System.out.println("Product inserted");

con.close();

    }
    catch(Exception e) {

        System.out.println(e);
    }
}
}

```

Retrieve Product Details

```

import java.sql.*;

public class DisplayProduct {

    public static void main(String[] args) {

        try {

            Class.forName("com.mysql.cj.jdbc.Driver");

            Connection con =
                DriverManager.getConnection(
                    "jdbc:mysql://localhost:3306/shop",
                    "root",
                    "root");

            Statement st = con.createStatement();

            ResultSet rs =
                st.executeQuery("SELECT * FROM product");

            while(rs.next()) {

                System.out.println(
                    rs.getInt("id") + " " +
                    rs.getString("name") + " " +
                    rs.getDouble("price"));
            }
        }
    }
}

```

```

        con.close();
    }
    catch(Exception e) {

        System.out.println(e);
    }
}
}

```

Important JDBC Classes

- **DriverManager** – establishes the database connection.
- **Connection** – represents the connection to the database.
- **Statement / PreparedStatement** – executes SQL commands.
- **ResultSet** – stores the result of a SELECT query.

Exception Handling

Database errors can occur due to incorrect SQL, wrong database details, or connection problems.

Therefore, JDBC code is placed inside:

```

try {
    // JDBC code
}
catch(Exception e) {
    System.out.println(e);
}

```

Code of Conduct:

I SRAVATHI K (192372127) (Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment. I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the student: SRAVANTHI K

RUBRICS FOR CALCULATION:

Criteria	Weightage (%)	Level 4 (Exemplary)	Level 3 (Proficient)	Level 2 (Developing)	Level 1 (Beginning)
Understanding of Scenario	20%	Demonstrates clear and complete understanding of the scenario and context	Shows good understanding with minor gaps	Partial understanding; misses key aspects	Misinterprets or fails to understand the scenario

Identification of Key Issues	20%	Accurately identifies all relevant issues and factors involved	Identifies most key issues with minor omissions	Identifies limited issues; some irrelevant points	Unable to identify key issues
Application of Concepts	25%	Applies appropriate concepts effectively to address the scenario	Applies concepts with minor errors	Limited or partially correct application	Unable to apply relevant concepts
Decision Making	20%	Makes well-reasoned, logical decisions with strong rationale	Makes reasonable decisions with some justification	Makes weak decisions with limited justification	No clear decisions made or rationale provided
Clarity of Response	15%	Response is clear, structured, and logically presented	Generally clear with minor issues	Somewhat unclear or poorly structured	Disorganized and difficult to understand